MalFSCIL: A Few-Shot Class-Incremental Learning Approach for Malware Detection

Yuhan Chai, Ximing Chen, Jing Qiu, *Senior Member, IEEE*, Lei Du, Yanjun Xiao, Qiying Feng, Shouling Ji, *Member, IEEE*, and Zhihong Tian, *Senior Member, IEEE*

***Abstract*—The continuous evolution of malware is posing a serious threat to personal privacy, enterprise data security, and global network infrastructure. For example, attackers can use phishing emails, botnets, etc. to induce victims to execute malware for nefarious purposes such as stealing sensitive information. Therefore, it is significant to develop effective and efficient methods to detect malware. Towards this, most state-of-the-art methods are focused on learning-based method. In order to adapt to the characteristics of sample scarcity and dynamic evolution of malware detection tasks, few-shot class incremental learning has been proposed as an efficient pairwise solution. Nevertheless, they still face two major challenges: 1) Catastrophic Forgetting: the erosion of existing knowledge by newly acquired knowledge during incremental learning. 2) Decision boundary confusion: after continuous multiple incremental sessions, the discriminative ability of the classification model is weakened. To address the above challenges, we propose a new Malware detection framework based on Few-Shot Class Incremental Learning, MalFSCIL, which utilizes a decoupled training strategy combined with a variational autocoder to mitigate catastrophic forgetting, and designs a dynamic boundary delineation method based on class prototyping to achieve accurate delineation of incremental decision boundaries. Extensive experimental results show that the proposed method outperforms the state-of-the-art techniques in malware detection and classification with high classification accuracy with open-source dataset and Internal enterprise dataset.**

***Index Terms*—Malware detection, few-shot class incremental learning, variational auto-encoder, graph attention networks.**

Received 10 May 2024; revised 10 October 2024; accepted 4 November 2024. Date of publication 12 December 2024; date of current version 20 March 2025. This work was supported in part by the National Science and Technology Major Project under Grant 2022ZD0119602, in part by the National Natural Science Foundation of China under Grant 62272114 and Grant U24A20336, in part by the Major Key Project of Peng Cheng Laboratory (PCL) under Grant PCL2022A03, in part by the Science and Technology Program of Guangzhou under Grant 2024A03J0399, in part by the China Computer Federation (CCF)-NSFOCUS "Kunpeng" Research Fund under Grant CCF-NSFOCUS202203 and Grant CCF-NSFOCUS2023003, in part by the Joint Research Foundation of Guangzhou University under Grant 202201020380, in part by Guangdong Higher Education Innovation Group under Grant 2020KCXTD007, in part by the Pearl River Scholars Funding Program of Guangdong Universities under Grant 2019, and in part by the 2024 University Research Projects of the Education Bureau of Guangzhou Municipality under Grant 2024312279. The associate editor coordinating the review of this article and approving it for publication was Dr. Luca Caviglione. *(Corresponding author: Jing Qiu.)*

Yuhan Chai, Ximing Chen, Jing Qiu, Qiying Feng, and Zhihong Tian are with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 510006, China (e-mail: chaiyuhan@gzhu.edu.cn; qiujing@gzhu.edu.cn; qiujing@gzhu.edu.cn; qiyingvon@gmail.com; tianzhihong@gzhu.edu.cn).

Lei Du is with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, Guangdong 518055, China, and also with the Peng Cheng Laboratory, Shenzhen, Guangdong 518066, China (e-mail: leidu_close@163.com).

Yanjun Xiao is with PINGXING Lab (Nsfocus Technology Group Company), Guangzhou, Guangdong 510663, China (e-mail: xiaoyanjun@nsfocus.com).

Shouling Ji is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China (e-mail: sji@zju.edu.cn).

Digital Object Identifier 10.1109/TIFS.2024.3516565

I. Introduction

MALWARE, with its advanced evasion techniques and constant evolution, poses a significant challenge to the field of cybersecurity. The challenges are predominantly in maintaining the confidentiality [1], integrity, and availability of systems. Additionally, the substantial economic impact of malware cannot be overlooked. According to a report [2], approximately 236.7 million ransomware attacks occurred globally in the first half of 2022, with an average economic loss of around $1.85 million per incident. To evade detection, malware developers have employed various obfuscation and hiding strategies, such as inserting dead code, reordering subroutines, and swapping code [3], [4]. These tactics not only enhance the stealthiness of malware but also increase the difficulty of traditional detection systems. Therefore, the research and development of advanced automated malware detection technologies hold significant practical importance and urgency.

Deep learning-based methods for malware detection have shown significant accuracy due to their ability to learn complex behavior patterns of malware from large datasets [5], [6]. By analyzing the behavior patterns, control flow graphs, and semantic features of malware within predefined datasets, these methods demonstrate exceptional generalization capabilities. However, the dynamic evolution of malware in the real-world cyberspace environment presents challenges to deep learning approaches. On one hand, a plethora of variants causes significant shifts in the behavioral patterns of malware samples (e.g., Emotet), leading to a deviation of old class sample features from the original training data and impacting the existing knowledge of the model. On the other hand, the emergence of zero-day and unknown new class malware makes it difficult for existing models to adapt and recognize these new threat patterns in a timely manner [7]. Particularly noteworthy is that in the early stages of new class samples, such as WannaCry, the data available for training is extremely limited, which not only increases the detection difficulty but also restricts the model's ability to promptly update and adapt to new threats.

Thus, exploring Few-Shot Class-Incremental Learning (FSCIL) algorithms for malware detection is of significant importance. This method integrates the principles of Few-Shot

Learning (FSL) and Class-Incremental Learning (CIL), providing a framework for real-world malware detection systems to swiftly adapt to the evolving nature of malware through learning from limited samples. While this approach offers the potential to address several challenges, it still faces two primary obstacles:

C1: Catastrophic Forgetting, which refers to the rapid loss of a model's memory of old classes while learning new ones. Conventional integrated model tuning of models not only leads to a decline in the representational ability of old knowledge but also cause overfitting when learning new classes. For instance, as described in [8], author utilized Hidden Markov Model to model API call sequences. When new classes malware are introduced, the model might exhibit a bias towards data from the new class, resulting in the forgetting of data pertaining to the old ones.

C2: Decision boundary confusion, as new classes are continually integrated, the decision boundaries between classes become increasingly complex and, in some cases, difficult to delineate effectively. For example, the incremental classification method proposed in [9] only considers a single round of incremental training, but its reliability in sustained incremental scenarios has yet to be adequately demonstrated.

In this paper, we primarily focus on detecting malware families and their associated Advanced Persistent Threat (APT) groups. To address the challenges outlined above, we proposed a Few-Shot Class-Incremental Learning framework, named MalFSCIL. MalFSCIL aims to mitigate the impact of integrated model tuning on the representation of old knowledge during the incremental training phase. To achieve this, we separate the feature extraction and classification during training sessions. In the base-class training stage, we introduce a Variational Autoencoder (VAE) to simulate the brain's memory replay mechanism, allowing for a thorough exploration of both new and old class knowledge. In the subsequent new class adaptation stage, we maintain the feature embedding parameters, thereby preserving the model's capacity to represent old classes. Furthermore, to resolve the challenge of decision boundary ambiguity inherent in incremental learning, MalFSCIL adopts a class-prototype-based strategy for defining inter-class boundaries. This approach is synergistically combined with a dynamic feature evolution graph and an additive angular margin loss function within graph attention networks, thereby ensuring precise and consistent boundary definition throughout the incremental learning process. By integrating reinforcement of prior knowledge with enhancements in the model's representational capacity, this approach aims to effectively mitigate the phenomenon of catastrophic forgetting. The main contributions of our work are as follows:

- We proposed a novel FSCIL framework for unknown malware detection, called MalFSCIL. This framework employed a decoupled training approach to consolidate previously acquired knowledge. During the base-class training stage, feature enhancement techniques are utilized to improve the representation of new knowledge, thus alleviating the effects of catastrophic forgetting.
- In the new class adaptation stage, we designed a continuously evolving classifier based on class prototypes. A composite objective function constraint is used to achieve both intra-class compactness and inter-class separability, effectively mitigating decision boundary confusion.
- Experiments conducted on two malware FSCIL datasets demonstrated that the proposed method significantly outperforms baseline methods.

II. Background & Related Work

A. Malware Detection

Traditional malware detection techniques can be broadly classified into static analysis [10], [3] and dynamic analysis [11], [12], [13]. In static analysis, Santos et al. [10] utilized the frequency representation of opcode sequences for malware detection and classification, demonstrating that data mining techniques can effectively uncover opcode correlations. Similarly, Cesare et al. [3] proposed a control flow based detection method using string feature vectors and the minimum matching distance to identify malware variants. Dynamic analysis, on the other hand, involves monitoring the actual execution of programs to detect malicious activities by analyzing their runtime behaviors. For instance, Lee et al. [11] employed n-gram analysis of API behavior data, enabling the detection of malicious activity through clustering techniques. Kim et al. [12] developed a method for malware detection based on API call sequences, while Cheng et al. [13] applied information retrieval theory to classify malicious executables, using a TF-IDF weighting scheme to calculate behavioral features. Sebastián and Caballero [14] proposed an open-source automated malware labeling tool that classifies samples based on category, malware family, file attributes, and behavioral characteristics. The tool is capable of continuously expanding new labels based on an open taxonomy to adapt to the evolving malware landscape. These approaches rely on predefined datasets for training malware detection models. Although they achieve relatively high accuracy, they are limited by their dependence on the training data. As a result, they can only classify malware belonging to the classes present in the training set. The dynamic nature of real-world data distribution over time can cause these models to perform poorly when confronted with new, previously unseen malware variants.

B. Learning-Based Malware Detection

The advancements in deep learning, coupled with enhanced computational power, have introduced new opportunities for detecting malware, allowing for the extraction of features directly from high-dimensional data. This marks a significant evolution in passive detection methods. Broadly, learning-based malware detection methods can be categorized into three main approaches: call sequence analysis [15], [16], [17], network traffic analysis [18], [19], [20], and visual analysis [21], [22], [23].

In the context of call sequence analysis, Alsulami et al. [15] pioneered the proposal of deep graph convolutional networks for detecting malicious behavior by analyzing API call sequences. Their method combines the temporal information of API call sequences with the spatial information of behavior

graphs, offering a novel perspective and technological pathway for malware detection. Zhang et al. [16] observed that while malware evolves, it often retains similar malicious semantics but may switch to new implementations using semantically equivalent APIs. Building on this observation, they designed APIGraph, which automatically extracts API knowledge from API documentation and incorporates this information into the training of malware detection models. Regarding network traffic analysis, Barut et al. [24] analyzed IP, HTTP, DNS, and unencrypted TLS record headers in network traffic, using these input byte sequences to propose an enhanced residual network model for classifying malware types and benign traffic. In terms of visual analysis, novel strategies have been explored for automatic malware detection. For instance, He et al. [21] developed plug-and-play attention mechanisms based on ResNeXt to capture the malware image channel's perception field of view. Yuan et al. [22] introduced a byte-level malware classification method utilizing Markov images. Mohammed et al. [23] suggested generating dual-graph grayscale images from binary executable files of malware using a dual-graph algorithm, which is then processed by improved convolutional neural networks for detection.

However, these approaches primarily focus on classifying samples from known malware categories, a limitation in real-world applications. Given the proprietary and customized nature of malware, training samples are often scarce. Moreover, the continuous emergence of unknown malware and variants results in the dynamic evolution of malware distribution, further exacerbating the limitations of traditional detection models. These factors make it difficult for conventional learning-based methods to effectively identify unknown or variant malware, thus limiting their practical performance.

C. Few-Shot Learning-Based Malware Detection

Few-Shot Learning (FSL) aims to develop models capable of being trained with only a small number of labeled samples, enabling the model to quickly adapt and accurately classify unseen data. Due to the dynamic and stealthy nature of malware, it is common that only a limited number of samples are available for analysis. As a result, some researchers have framed malware detection as a challenge for FSL, which typically employs either optimization-based or metric-based learning strategies. Optimization-based methods focus on rapidly training a model to handle new tasks with minimal training data [25]. A2-CLM [26] represented malware behaviors as heterogeneous graphs and generates simulated samples through various adversarial attacks, such as attribute masking and obfuscation, to improve model robustness. On the other hand, metric-based approaches learn effective embedding functions for feature extraction and apply suitable distance metrics to measure the similarity between support and query instances [27]. Tang et al. [28] developed the ConvProtoNet model for few-shot malware classification, while Wang et al. [29] proposed a dynamic analysis-based method for the same purpose. These methods have demonstrated strong performance in detecting malware with limited samples. However, the rapid evolution of attack strategies, including the emergence of numerous zero-day vulnerabilities and novel attack techniques, poses significant challenges for FSL-based algorithms. These approaches often struggle to keep pace with the continually shifting threat landscape, limiting their effectiveness in addressing new and evolving forms of malware.

D. Class Incremental Learning-Based Malware Detection

Class Incremental Learning (CIL) enables models to incrementally learn and integrate new classes of information while retaining knowledge of previously learned classes. This approach is particularly useful in dynamically changing data environments. Hinton et al. [30] employed knowledge distillation to facilitate the incremental online updating of models, allowing them to learn the features of new classes without forgetting previously acquired knowledge [31]. To address the issue of catastrophic forgetting in incremental learning, Li et al. [32] introduced an incremental malware detection framework based on multi-class support vector machines, which adds new classification layers and updates existing ones to improve classification performance. DOMR [33] proposed an episode-based representation learning approach to mitigate catastrophic prediction errors and representation drift in identifying Android malware families. Other domains have also seen a significant focus on addressing catastrophic forgetting in CIL [34], [35]. In this context, Hu et al. [35] introduced a causal framework to explain catastrophic forgetting in CIL. However, these methods generally assume that ample training data is available for new classes. This assumption does not align with the reality of malware detection, where the scarcity of malware samples can lead to overfitting when learning new class knowledge. As a result, this exacerbates the problem of catastrophic forgetting, further limiting the effectiveness of CIL-based methods in real-world malware detection scenarios.

E. FSCIL-Based Malware Detection

FSCIL is an emerging research area first introduced by Tao et al. [36], with the goal of enabling models to continuously learn in real-world scenarios despite evolving data distributions and limited labeled samples [37]. Methodologically, TOPIC [36] employs a growing neural gas network [38] to model the feature space topology, preserving existing knowledge while incorporating new classes. Instead of retraining the entire model, FSLL [39] was proposed to selecting a subset of model parameters for each incremental session. Additionally, SPRR [40] enhances feature representation scalability, ensuring adaptability across different simulated incremental learning processes. To more effectively retain old knowledge, Cheraghian et al. [41] introduced semantically informed embedded words in FSCIL.

In the domain of malware detection, the application of FSCIL is still in its early stages. IMC [9] introduced by Qiang et al., was one of the first frameworks to classify unknown malware classes with high accuracy while maintaining the ability to identify known malware. However, this approach is limited by its focus on class growth within a single incremental session, which does not adequately address the continuous evolution required for malware detection. Although FSCIL has demonstrated promising results in fields such as image

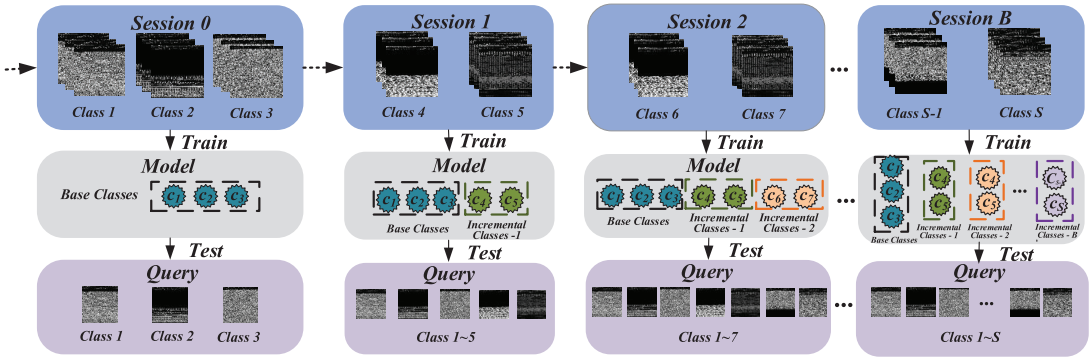

Fig. 1. Few-shot class-incremental learning setting for malware detection.

recognition [42], [43], [44] and vehicle recognition [45], these methods generally suffer from a gradual decline in recognition performance as the incremental rounds progress. This degradation stems from the inability to effectively manage decision boundary confusion and catastrophic forgetting, thereby compromising the models' long-term learning capacity and stability.

To address these challenges, we proposed a computer vision-based malware detection framework, MalFSCIL, which divides the learning process into two stages: "old knowledge reshaping" (base class training stage) and "new knowledge increment" (new class adaptation stage). By decoupling the training process and improving the feature extraction module, the model's representational ability is enhanced while reinforcing existing knowledge. Moreover, in the new class adaptation stage, we adopt a prototype-based boundary delimitation strategy to dynamically adjust decision boundaries, enabling the model to continuously update and learn new classes.

III. Problem Definition

Zero-day attacks represent a significant threat to network security, as they can bypass existing security measures undetected. These unknown attacks easily evade current protective mechanisms, posing substantial risks to systems. However, due to the insufficient or limited availability of attack samples, traditional models struggle to effectively adapt to the increasing variety of such threats. To address these limitations, this paper investigates an incremental learning strategy that does not rely on prior knowledge of attack samples. Instead, it focuses on extracting essential knowledge from previously trained models, which is then optimized and retained in new models. For the detection task, we consider a more practical FSCIL malware detection scenario, which includes a base session followed by multiple incremental sessions. In the base session, a sufficient number of samples is available to train a primary model. In each subsequent incremental session, knowledge of new malware classes is acquired from only a few labeled samples. Once in an incremental session, previous training samples are no longer accessible, meaning the model must retain knowledge from earlier sessions while incorporating new information. The specific problem definition is outlined as follows:

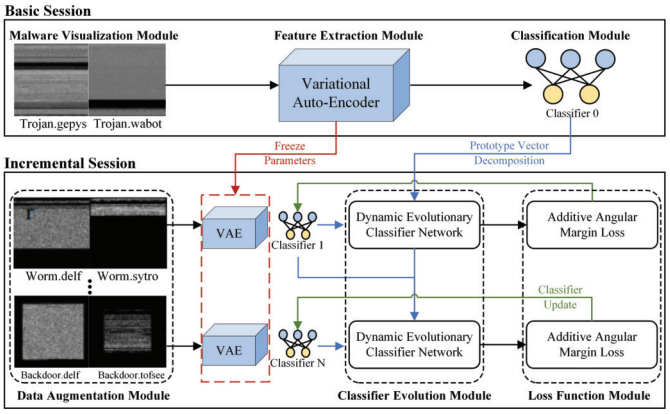

Fig. 2. Framework of MalFSCIL.

As shown in Fig. 1, $\{D^0_{train}, D^1_{train}, \ldots D^B_{train}\}$ and $\{D^0_{test}, D^1_{test}, \ldots D^B_{test}\}$ represent the training set and test set of different sessions, respectively, where B denotes the total number of sessions. The label space corresponding to the training set D^b_{train} is denoted as Y^b. Specifically, there is no overlap between the malware classes across different sessions, meaning that for arbitrary b and b', we have:

$$Y_b \cap Y_{b'} = \emptyset, \quad b \neq b'$$

Typically, in the first session, D^0_{train} is a relatively large dataset with sufficient data to train the base model, while the dataset $D^{b>0}_{train}$ in subsequent sessions contains a small number of classes, with each class having only a few training samples. These limited data are organized in an N-way K-shot format, *i.e.*, meaning that N classes, and only K samples per class are available. Formally,

$$D^{b>0}_{train} = \{(x_i, y_i)\}_{i=1}^{NK}$$

In each incremental session b, only the current training set D^b_{train} is used for model training. Following the training phase in session b, the model is evaluated on all the classes encountered up to that point. Thus, the test dataset for session b denoted as D^b_{test} contains the test data of all previous and current classes, with a label space $\{Y_0, Y_1, \ldots Y_b\}$.

IV. Methodology

Fig. 2 shows the the overall design of the proposed MalFSCIL framework. We first present an overview of the

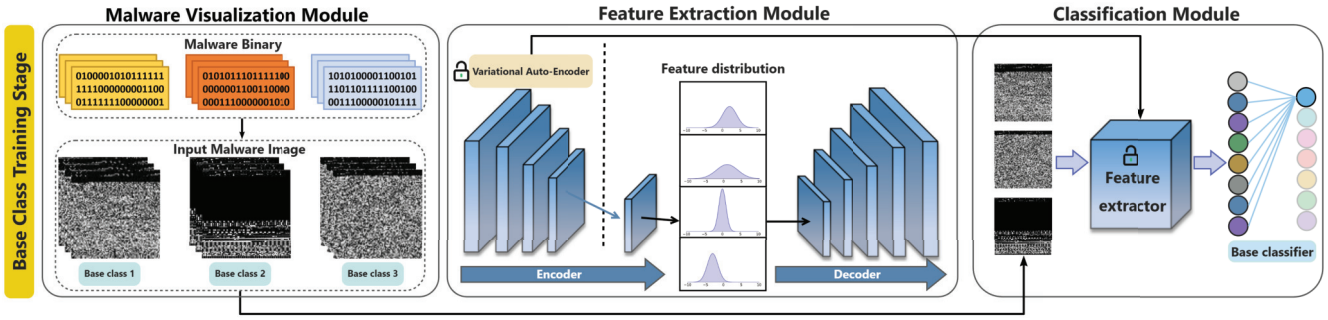

Fig. 3. Diagram for base class training stage.

framework, followed by a detailed explanation of its two core stages: the base class training stage and the new class adaptation stage.

A. Overview

In response to the emergence of unknown malware variants, this paper introduces a few-shot class-incremental learning (FSCIL) framework, consisting of two key stages: the base class training stage and the new class adaptation stage. During the base class training stage, an initial model is constructed using sufficient labeled data. The new class adaptation stage focuses on continuously updating the model with only a few samples from newly emerging classes. However, the continual integration of new classes may lead to catastrophic forgetting, which significantly diminishes the model's performance. Given the critical need for precision in malware detection, the system must efficiently classify malware types and execute precise mitigation strategies against diverse malicious threats. Thus, we address Challenge C1, catastrophic forgetting, by adopting a combined approach that emphasizes both the preservation of old knowledge and the acquisition of new knowledge.

This section introduces a decoupled model training framework aimed at preserving previously acquired knowledge. The model is divided into two primary components: a feature extractor F and a classifier C. In the base class training stage, conventional neural network training is performed using a large-scale labeled dataset $D_0 = \{(x_i, y_i)\}$, where x_i represents the malware samples and y_i corresponds to the malware type labels. Initially, an enhanced feature extractor F is trained in an unsupervised manner to learn a mapping $f : X \to Z \to X$, which maps the feature space X into another space Z and restores itself. A linear classifier C is then trained to detect and classify malware belonging to the base classes.

In the new class adaptation stage, the feature extractor F is reused, and its parameters are frozen to preserve the original feature representation learned during the base class training stage. This prevents unknown samples from compromising the model's feature representation during incremental learning. A continuously evolving classifier C_{new} is then trained using only a few malware samples from the new class dataset $D_{new} = \{(x_j, y_j)\}$. To further address catastrophic forgetting, we propose an inter-class boundary delimitation method based on class prototypes.

While the decoupled training framework helps mitigate catastrophic forgetting to a certain extent, it is not entirely sufficient to adapt to the representation of new knowledge. Therefore, a VAE-based (Variational Autoencoder) feature extractor will be introduced in the following subsection to further enhance knowledge acquisition and reduce catastrophic forgetting.

B. Base Class Training Stage

The base class training stage for malware detection is typically conducted offline, where the model is trained on a sufficient number of samples to accurately identify all known types of malware. The architecture of this stage is depicted in Fig. 3, and consists of three key components: the *Malware Visualization Module*, the *Feature Extraction Module*, and the *Classification Module*.

1) Malware Visualization Module: MalFSCIL employs a machine vision technique to convert malware binary executables into greyscale images, enabling the model to capture the malware's instructions, data structures, and code logic as visual textures, patterns, or frequency distributions. This abstraction allows the model to identify underlying patterns within the binary code. Specifically, this process involves interpreting the malware's binary executable as an 8-bit unsigned integer vector, which is then transformed into a two-dimensional array. The resulting array is subsequently mapped to a greyscale spectrum ranging from $[0, 255]$, where 0 represents black and 255 represents white.

2) Feature Extraction Module: In the feature extraction module, a VAE is employed to encode the malware's greyscale images, enhancing the feature extraction process as previously mentioned. VAE [46], a form of deep generative model, describes the potential space of the data in terms of probability distributions, unlike the traditional autoencoder (AE), which works with discrete values. By capturing key elements from the input data, the VAE generates new instances from these latent features, removing noise through repeated mapping between the latent space and data space. This process mirrors the memory encoding and retrieval functions of the hippocampus in the human brain [47].

From our perspective, this approach improves data diversity and reduces the risk of overfitting when working with limited samples. Additionally, it helps the model retain knowledge

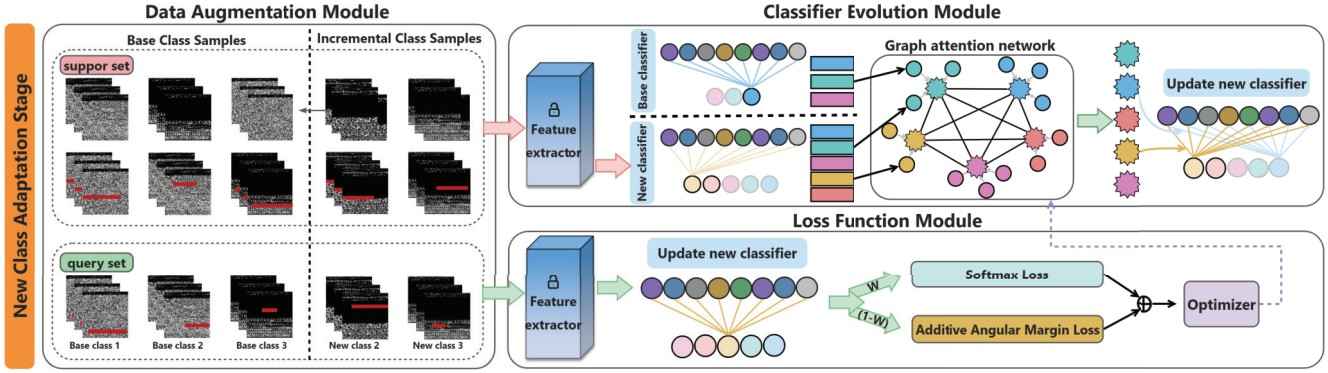

Fig. 4. Diagram for new class adaptation stage.

from previous sessions while learning new features, thereby mitigating catastrophic forgetting.

Specifically, the VAE consists of an encoder $q(z|x)$ which serves as the feature extractor, and a decode $p(x|z)$, responsible for reconstructing the input. Here, x represents the sample data, and z is the latent variable capturing the features of x, expressed as $z = \mu + \sigma\Theta\varepsilon$, where μ and ε are derived from the encoder network, and ε is a variable from the standard normal distribution $N(0, I)$. The encoder's goal is to simulate an approximate posterior distribution $q(z|x) \approx p(x|z)$, generating a latent vector z given an input sample x. The decoder models distribution $p(x|z)$ to regenerate the real sample x from the latent vector $z \sim q(|x)$ VAE assumes that $p(x|z)$ follows a Gaussian. Thus, the encoder generates the mean and variance for each sample x, which are used to construct an approximate posterior $q(z|x)$. In turn, the decoder reconstructs the input data from the latent vector.

The VAE is trained in an unsupervised manner using the base class dataset. The loss function for the VAE is formulated as:

$$L_{VAE} = -E_{Z\sim q(z|x)}[log\ p(x|z)] + D_{KL}[q(z|x)\|p(z)]. \quad (1)$$

In this equation, the first term represents the reconstruction loss, encouraging the decoder to generate samples as close as possible to the original inputs. VAE is trained end-to-end using stochastic gradient descent to minimize the reconstruction error between the generated and original data. The second term, the Kullback-Leibler (KL) divergence, acts as a regularizer, measuring the difference between the approximate posterior distribution $q(z|x)$ and the latent vector space probability distribution $p(z)$. The smaller the KL divergence, the closer the two distributions are. By combining this VAE-based feature extractor with the decoupled training framework, we can more effectively extract latent features from malicious samples and enhance knowledge acquisition. This approach significantly mitigates catastrophic forgetting, while simultaneously preserving old knowledge.

3) Classification Module: The final component of the framework is a fully connected network classifier, designed to efficiently interface with the feature extractor and support the subsequent incremental training stage. During model training, a co-training strategy is employed to perform supervised learning on the base class dataset, where both the feature extractor and classifier are trained simultaneously.

To evaluate the model's performance on complex malware detection tasks, a comprehensive loss function is introduced. This loss function comprises two key components. The first is the cross-entropy loss, which measures the discrepancy between the model's predicted output and the actual malware labels. The cross-entropy loss is expressed as:

$$L_{CE} = -\frac{1}{N}\sum_{i=1}^{N} log\frac{e^{W_{y_i}^T x_i + b_{y_j}}}{\sum_{j=1}^{n} e^{W_j^T x_i + b_j}}. \quad (2)$$

where, x_i and y_i represent the depth feature and the corresponding label of the $i-th$ sample, respectively; $W_j \in \mathbb{R}^d$ represents the $j-th$ column of the weight $W_j \in \mathbb{R}^{d\times n}$; and $b_j \in \mathbb{R}^n$ represents the bias term.

The second component of the loss function is the reconstruction loss from the VAE, as described earlier. By combining these two loss functions, the model achieves end-to-end optimization during training. The total training loss on the base class training set D_{train}^0 is given by:

$$L_{total} = L_{CE} + L_{VAE}. \quad (3)$$

C. New Class Adaptation Stage

In the new class adaptation stage, the emergence of new malware types with limited samples presents a significant challenge to the decision boundaries established by the original classifier trained on the base class dataset. To effectively address the issue of sample sparsity and to resolve the problem of unclear decision boundaries, we introduce two key modules: the *Classifier Evolution Module* and the *Loss Function Module*. The diagram illustrating the new class adaptation stage is shown in Fig. 4.

1) Classifier Evolution Module: To continuously adapt to new classes in the feature space, an intuitive approach is to gradually expand the decision boundaries to meet classification requirements. However, the ongoing introduction of new malware classes during incremental learning can lead to decision space obfuscation, particularly after multiple rounds of learning. This results in imprecise boundary delineation in the high-dimensional feature space. To address this challenge, inspired by [48], we employ a Graph Attention Network (GAT) [49] to fine-tune the weights of classifier prototypes

accumulated during individual incremental sessions. At the global task level, weighting individual classifier prototypes enables the transfer of contextual information between old and new classes. This process enhances decision boundaries across the entire class spectrum by leveraging the GAT's focus on distinguishable feature representations, improving the classifiers' detection performance. Specifically, the training set D^b_{train} is represented as the dataset for the b-th round of incremental sessions. We train a fully connected classifier for all classes seen so far, while freezing the parameters of the feature extractor. The neural network weight matrix is defined as $P_b = \{p^0_b, p^1_b, \ldots, p^c_b\}$, where p^c_b denotes the mapping vector of the classifier from the input features to the output probability of class c in the b-th incremental session.

During the training of the b-th incremental session, we collect the prototype vectors for all classifiers from both the previous and current rounds, represented as:

$$\mathcal{P} = \begin{pmatrix} p^0_0 \cdots p^{c_1}_0 & & \\ p^0_1 \cdots p^{c_1}_1 & \cdots & p^{c_2}_1 \\ \vdots \quad \cdots \quad \vdots & \cdots & \vdots \\ p^0_b \cdots p^{c_1}_b & \cdots & p^{c_2}_b \cdots p^{c_b}_b \end{pmatrix} \tag{4}$$

Each element of the matrix $\mathcal{P}$ represents the classifier prototype vector corresponding to class c in each session. In a dynamic evolutionary classifier module, each prototype vector is treated as a node, and the nodes are connected to form a heterogeneous graph. The heterogeneous graph is then fed into the GAT model to update the classifier prototypes. For a prototype vector p_j of node j, we calculate the relationship coefficient with all nodes, such as the relationship between node j and node l, expressed as:

$$r_{jl} = sim(\phi(p^j), \varphi(p^l)) \tag{5}$$

Here, ϕ and φ are linear transformation functions that map prototype vectors to a new metric space, and $sim(a, b)$ is a similarity function. The relationship coefficients are then normalized to derive the attention weight a_{jl} for the central node j with respect to node l:

$$a_{jl} = \frac{\exp(r_{jl})}{\sum_{v=1}^{|\tilde{P}_B|} \exp(r_{jv})} \tag{6}$$

Using the normalized attention weight, the information from all nodes in the graph is aggregated and fused with the original prototype vector. The updated prototype vector of node j is:

$$p'_j = p_j + \left(\sum_{l=1}^{|\tilde{P}_B|} a_{jl} p_l \right) \tag{7}$$

Finally, an incremental classifier is constructed for all classes based on the updated prototype vectors. To optimize the GAT model further, we introduce an objective function constraint using an additive angular margin loss function. This function aims to achieve intra-class compactness and inter-class separability of features, guided by the optimization goal.

2) Loss Function Module: To further refine the decision boundaries between different classes of prototypes, we augment the model with an additive angular margin loss function [50]. This approach not only ensures accurate classification but also emphasizes intra-class compactness and inter-class separation. Our loss function is a weighted combination of two components. The first component is the softmax loss function, which aims to classify the data correctly:

$$L_{softmax} = -\frac{1}{N} \sum_{i=1}^{N} log \frac{e^{W^T_{y_i} x_i + b_{y_i}}}{\sum_{j=1}^{n} e^{W^T_j x_i + b_j}} \tag{8}$$

where $W = \frac{W*}{\|W*\|}$ and $x = \frac{x*}{\|x*\|}$. Here, N represents the number of training samples, x_i is the feature vector of the $i-th$ sample, and y_i is the true label of the $i-th$ sample. $W_j \in \mathbb{R}^d$ represents the $j-th$ column of the weight $W_j \in \mathbb{R}^{d \times n}$, which is the weight vector of the $j-th$ class, and $b_j \in \mathbb{R}^n$ is the bias term. The batch size and number of classes are N and n, respectively.

However, relying solely on the softmax loss may overlook intra-class compactness and inter-class separation. To address potential feature confusion and overlap in incremental learning, we introduce an angular margin loss function. This function serves as a constraint to maximize the separation of classification boundaries in angular space:

$$L_{arc} = -\frac{1}{N} \sum_{i=1}^{N} log \frac{e^{s(cos(\theta_{y_i} + m))}}{e^{s(cos(\theta_{y_i} + m))} + \sum_{j=1, j \neq y_i}^{n} e^{scos\theta_j}} \tag{9}$$

In this equation, $cos\theta_j = W^T_j x_i$, s is the scaling factor, m is the angular margin penalty, and θ_j is the angle between the weight W_j and the feature vector x_i.

In summary, our total loss function is a weighted combination of these two losses:

$$L_{as} = \omega_v * L_{arc} + (1 - \omega_v) * L_{softmax} \tag{10}$$

where ω_v is a weight coefficient. This combined loss strategy ensures that the model not only classifies the data accurately but also distinctly separates different classes in the feature space.

3) Incremental Learning Process: During model training, the emergence of incremental sessions poses a challenge, as regular datasets may not be sufficient to support continuous incremental training. To address this and simulate a realistic malware detection scenario with few-shot incremental learning, we adopt a meta-learning framework to construct fake-base and fake-incremental tasks in the base dataset. Specifically, we extract N_0 classes from the initial dataset D'_{train} as fake-base classes, and randomly select K_0 samples from each malware class to construct the fake-base class support set (training set) S_0. In addition, W_0 samples from the remaining data are drawn to form the fake-base class query set (test set) Q_0. The feature extractor θ_f and linear classifier θ_{c0}, which capture the potential features of the samples, are obtained through joint optimization during the base class training stage.

A similar process is applied to form the fake-incremental adaptation stage. Here, N_i classes are extracted from the dataset as fake-incremental classes for each incremental session. The number of samples is first extended using random masking techniques, and the augmented samples are then

TABLE I
DETAILS AND INCREMENTAL PARTITIONING OF BODMAS DATASETS AND NSFOCUS REAL-WORLD MALWARE DATASETS

Dataset	Base session		Incremental session		
	#Classes	#Samples	#Classes	#Samples	Incremental Form
Incremental Few-shot Bodmas	31	500	50	5	5-Way 5-Shot
NSFOCUS Real-World Malware	25	≥ 117	80	5	10-Way 5-Shot

Algorithm 1 Incremental Learning Process

Input: Malware datasets D_0; GAT model θ_G; feature extractor θ_f
Output: Continuously evolving malware classifier θ_C

Convert the binary executable in D_0 into a grey scale images dataset D_0'.
for *each session* i **do**
  if $i = 0$ **then**
    $\{\mathcal{S}_0, \mathcal{Q}_0\} \Leftarrow$ Sample the support and query set from D_0' as the fake base dataset;
    $\theta_f \Leftarrow$ Update feature extractor with $\{\mathcal{S}_0, \mathcal{Q}_0\}$ by eq. (4);
  else
    $\{\mathcal{S}_i, \mathcal{Q}_i\} \Leftarrow$ Sample the support and query set from D_0' as the fake incremental dataset;
    $\theta_{Ci} \Leftarrow$ Learn classifier with $\{\mathcal{S}_i\}$ by eq. (10);
    $\tilde{p}_i \Leftarrow$ separating classifier into each class's prototype vector;
    $\theta_{Ci} \Leftarrow$ Update prototypes using $\theta_G(\tilde{p}_i)$ by eq. (5,6,7);
    $y_i \Leftarrow$ Use θ_C predict query set $\mathcal{Q}_i$;
    $\theta'_{ci} \Leftarrow$ Optimize θ_G by $\theta_G - \nabla_\theta(\mathcal{L}(y_i, y'_i))$;
  end
end

divided into a support set S_i and a query set Q_i. A classifier network θ_{ci} is pre-trained on the support set using frozen feature extractors. The classifiers θ_{ci} from all current incremental sessions, along with the base classifiers θ_{c0}, are decomposed into prototype vectors and injected into a GAT model. This model aggregates similar prototype features and updates the classifier prototypes to generate a new classifier prototype vector p_i. Finally, by combining p_i into a new classifier and fine-tuning it with the query set Q_i, the resulting unified classifier becomes capable of accurate malware classification across a wide range of classes. In summary, Algorithm 1 provides an overview of the entire incremental learning process.

V. EXPERIMENT

A. Experimental Setup

1) Dataset: In this section, we introduce two malware datasets and the methods used to adapt them to few-shot incremental learning scenarios for malware detection.

a) Incremental Few-Shot Bodmas: BODMAS dataset [51] contains 57,293 malware samples collected ranging from August 2019 to September 2020, with carefully curated family information. As shown in Table I, the Incremental Few-shot Bodmas dataset is derived from BODMAS and consists of 81 classes. 31 classes are designated as base classes, with each class containing 500 samples. The remaining 50 classes are incremental classes, with 75 samples per class. To evaluate the classification performance across different few-shot incremental settings, the 50 incremental classes are further divided into 10 sessions, with each session containing 5 classes and 5 training samples per class. Each malware sample is converted into an image of size 84×84.

b) Nsfocus Real-World Malware: The NSFOCUS Real-World Malware dataset was provided by NSFocus [52] and consists of real-world malware samples collected before 2022, all of which have undergone comprehensive security analysis. The samples are annotated based on their association with Advanced Persistent Threat (APT) groups, involving malware commonly used by various APT organizations (e.g., Shamoon, APT29, and Red October). This dataset contains over 10,000 samples, primarily targeting the Windows platform. The data was collected by NSFocus's professional security team from real-world network environments, ensuring the authenticity and relevance of the samples. The dataset consists of 105 classes, with 25 classes used as base classes and 80 classes designated as incremental classes. The 80 incremental classes are equally divided into 10 sessions, with each session containing 8 classes and 5 training samples per class.

Specifically, both datasets are exclusively composed of malicious samples, thus directing the focus of this study toward the detection and classification of malware families and malware APT groups. For a more comprehensive understanding of the datasets, Fig. 6 presents the data distributions for *Incremental Few-shot Bodmas* and *NSFOCUS Real-World Malware* datasets, respectively.

2) Evaluation Metrics: The proposed method was comprehensively evaluated using four key metrics:

a) Accuracy in Each Session: After each session, the model's performance was evaluated using the test set, and the $Top1$ accuracy is reported in this paper. A successful prediction occurs when the predicted label matches the ground truth label. The $Top-1$ accuracy after the i-th session is expressed as A_i, with higher A_i indicating better prediction accuracy.

b) Performance Degradation (PD): . To assess the model's ability to retain previously learned knowledge, we measured the performance degradation rate (PD). This is defined as $PD = A_0 - A_B$, where A_0 represents the accuracy after the base session and A_B represents the accuracy after

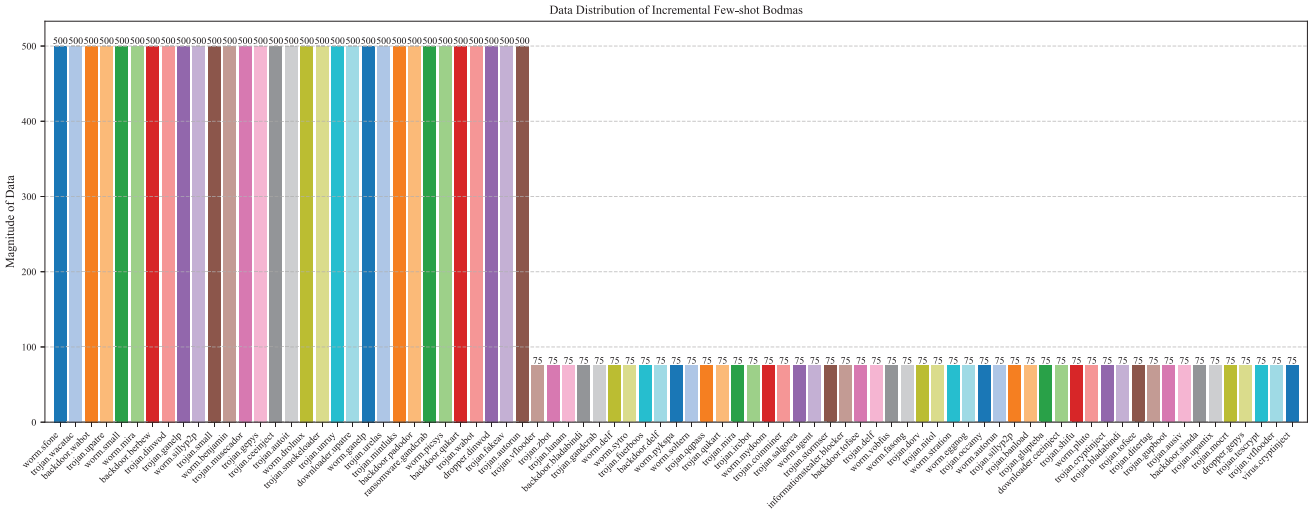

Fig. 5. Data distribution of incremental few-shot bodmas.

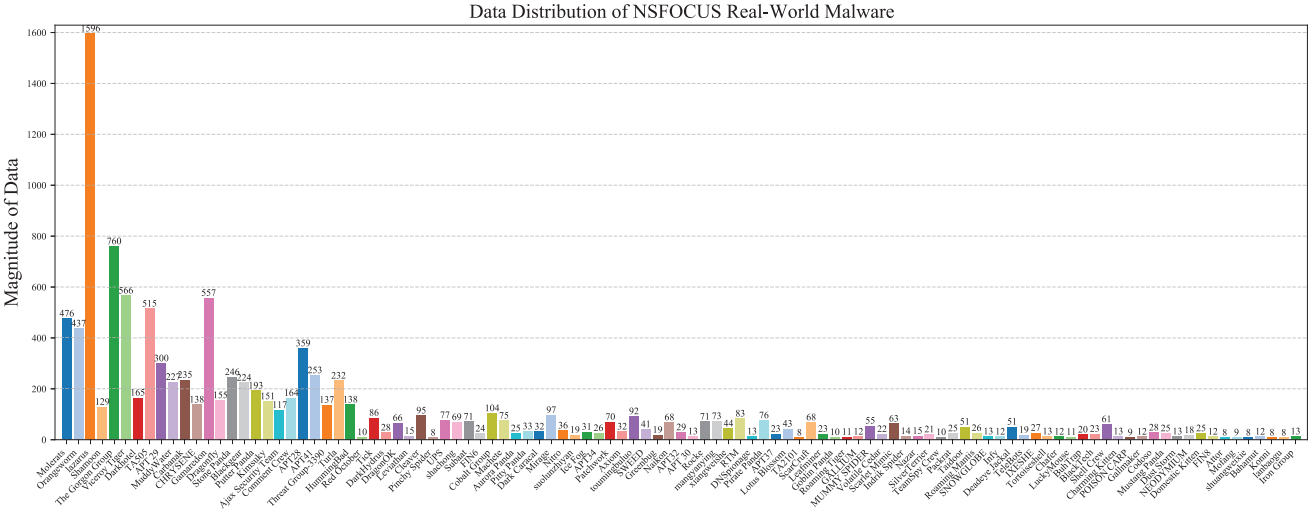

Fig. 6. Data distribution of NSFOCUS real-world malware.

the final incremental session. Lower performance degradation rates indicate less forgetting of previously learned information.

c) Average Accuracy: We calculated the average accuracy (*AverageAcc.*) across all sessions, as follows $\sum_{i=1}^{B} \frac{A_i}{B}$. This metric provides an overall measure of performance across the incremental learning process.

d) Final Session Improvement: Additionally, we measured the performance improvement of the proposed method in comparison with different baselines after the final session of the incremental process.

e) Area Under Time (AUT): To quantify the performance variation of a model across incremental sessions, AUT [53] integrates performance metrics over multiple incremental sessions, effectively capturing the model's overall stability and its resilience against catastrophic forgetting. The calculation is expressed as $AUT(\mathbb{P}, N) = \frac{1}{N-1} \sum_{k=1}^{N-1} \frac{\mathbb{P}(X_{k+1}) + \mathbb{P}(X_k)}{2}$.

3) Baseline: The baseline method, referred to as "finetune," involved fine-tuning the model using few-shot instances. We compared our method with several classical Class-Incremental Learning (CIL) methods, including iCaRL [54] and FOSTER [55]. Moreover, we compared the performance of our method against the state-of-the-art FSCIL methods: GP-Tree [44], C-FSCIL [56], CEC [48], FACT [42], and BFS-NID [57].

4) Experimental Details: We used ResNet18 [58] as the backbone for experiments on both the Incremental Few-shot Bodmas and NSFOCUS Real-World Malware datasets. The proposed network was implemented using the PyTorch library, and Stochastic Gradient Descent (SGD) with momentum was employed for optimization. The graph model was trained for 50 iterations with a learning rate of 0.0002, which was decayed by 0.5 after every 20 iterations. Additionally, ResNet18 was used as the encoder and a transposed convolutional network as the decoder for training the VAE.

B. Comparison With the State-of-the-Art Methods

Accuracy is an important metric for evaluating the effectiveness of malware detection tasks. High accuracy not only reduces the workload of secondary verification by experts but also allows for the timely detection of malicious attacks

TABLE II

COMPARISON OF THE PROPOSED METHOD WITH STATE-OF-THE-ART METHODS ON INCREMENTAL FEW-SHOT BODMAS DATASET† INDICATES RESULTS BASED ON PERFORMANCE TESTING USING THE PUBLICLY AVAILABLE SOURCE CODE OF THE RESPECTIVE PAPER

Method	Accuracy in each session (%)											PD	Average Acc	AUT
	0	1	2	3	4	5	6	7	8	9	10			
Finetune†	90.19	86.36	83.43	79.91	77.69	74.93	72.62	70.24	68.07	65.89	64.07	26.12	75.76	69.86
iCaRL [54]†	90.29	86.67	84.60	80.66	79.83	77.15	74.52	70.14	68.66	67.01	65.79	24.50	76.85	71.62
FOSTER [55]†	90.39	86.57	85.85	79.31	76.31	76.43	73.30	69.51	69.90	67.64	66.37	24.02	76.51	72.14
GP-Tree [44]†	88.16	83.32	80.27	77.27	76.00	74.15	73.33	71.62	69.37	67.34	66.81	21.35	75.24	72.53
CFSCIL [56]†	88.84	86.95	85.52	81.04	79.83	79.41	78.39	76.10	75.37	74.67	74.69	14.15	80.07	78.50
CEC [48]†	89.10	85.91	85.79	81.29	80.33	80.11	78.67	77.05	75.93	74.56	75.57	13.53	80.39	79.19
FACT [42]†	90.10	87.00	86.27	82.45	81.14	80.66	79.85	77.48	76.76	74.98	76.54	14.46	81.12	79.83
BFS-NID [57]†	89.84	86.67	84.24	84.04	82.78	81.50	79.80	78.00	76.37	75.09	74.27	15.57	81.33	79.65
MalFSCIL	**90.58**	**87.85**	**87.65**	**84.27**	**83.53**	**82.63**	**81.51**	**79.49**	**78.90**	**78.05**	**78.25**	**12.53**	**82.97**	**82.32**

TABLE III

COMPARISON OF THE PROPOSED METHOD WITH STATE-OF-THE-ART METHODS ON THE NSFOCUS REAL-WORLD MALWARE DATASET. † INDICATES RESULTS BASED ON PERFORMANCE TESTING USING THE PUBLICLY AVAILABLE SOURCE CODE OF THE CORRESPONDING PAPER

Method	Accuracy in each session (%)									PD	Average Acc	AUT
	0	1	2	3	4	5	6	7	8			
Finetune†	72.77	66.18	61.07	57.05	53.81	51.67	50.20	49.28	48.18	24.59	56.69	49.53
iCaRL [54]†	72.65	65.97	60.33	56.54	52.37	51.16	47.34	46.50	44.78	27.87	55.29	48.54
FOSTER [55]†	75.35	68.54	62.26	58.81	54.42	53.27	51.27	50.00	49.32	26.03	58.14	50.80
GP-Tree [44]†	70.95	59.23	49.93	41.13	38.37	35.29	32.47	32.02	27.97	42.98	43.04	42.10
CFSCIL [56]†	77.00	68.86	62.02	52.18	48.50	46.98	45.79	44.08	42.43	34.57	54.20	57.79
CEC [48]†	78.17	68.48	64.08	60.51	58.20	56.41	54.64	52.67	53.33	24.84	60.72	62.25
FACT [42]†	78.33	68.96	64.91	60.06	57.31	55.33	53.10	51.26	50.90	27.43	60.02	60.02
BFS-NID [57]†	76.39	64.40	56.05	51.99	49.40	47.02	43.24	40.68	40.13	36.26	52.14	53.10
MalFSCIL	**78.28**	**69.78**	**64.56**	**61.21**	**58.86**	**56.93**	**55.39**	**54.09**	**54.19**	**24.09**	**61.48**	**63.57**

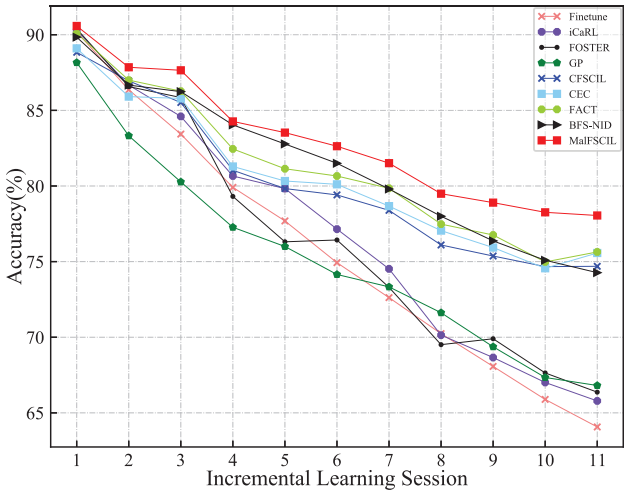

Fig. 7. Test accuracy versus the Incremental session for the Incremental Few-shot Bodmas dataset with 5-Way 5-Shot incremental scenario.

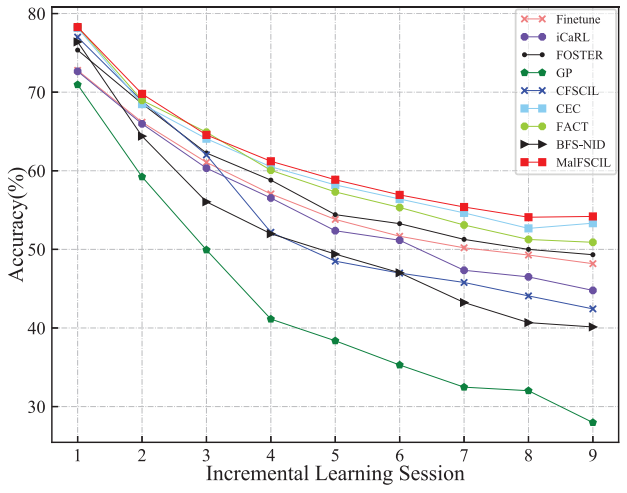

Fig. 8. Test accuracy versus the Incremental session for the NSFOCUS real-world malware dataset with 10-Way 5-Shot incremental scenario.

in high-throughput cyberspace. Accurate classification results enable the system to adopt appropriate defensive strategies against threats. However, malware detection performance can be hindered by several factors, including variations in coding styles and the use of obfuscation techniques, which lead to large intra-class variations and narrow inter-class differences. This challenge becomes even more significant in continuous class-incremental learning scenarios. Most detection models encounter challenges in tackling issues like catastrophic forgetting and decision boundary obfuscation, thereby impeding their ability to perform effectively in extended incremental sessions.

To address these challenges, we conducted extensive experiments on two datasets: the 5-Way 5-Shot scenario on the Incremental Few-shot Bodmas dataset and the 10-Way 5-Shot scenario on the NSFOCUS Real-World Malware dataset. The results are visualized in Fig. 7 and Fig. 8, and the corresponding numerical results are presented in Tables II and III, respectively.

TABLE IV
ABLATION STUDY OF OUR MALFSCIL ON INCREMENTAL FEW-SHOT BODMAS

Method		Accuracy in each session (%)											PD	Average Acc	AUT
VAE	ArcFace	0	1	2	3	4	5	6	7	8	9	10			
×	×	89.10	85.91	85.79	81.29	80.33	80.11	78.67	77.05	75.93	74.56	75.57	13.53	80.39	79.19
✓	×	90.52	87.55	87.35	84.15	83.03	81.92	80.82	78.99	78.22	77.61	77.46	13.06	82.51	81.65
✓	✓	90.58	87.85	87.65	84.27	83.53	82.63	81.51	79.49	78.90	78.26	78.05	12.53	82.97	82.32

TABLE V
ABLATION STUDY OF OUR MALFSCIL ON NSFOCUS REAL-WORLD MALWARE DATASET

Method		Accuracy in each session (%)									PD	Average Acc	AUT
VAE	ArcFace	0	1	2	3	4	5	6	7	8			
×	×	78.17	68.48	64.08	60.51	58.20	56.41	54.64	52.67	53.33	24.84	60.72	62.25
✓	×	78.61	69.76	63.74	60.89	58.65	56.69	55.11	53.09	53.95	24.66	61.17	63.42
✓	✓	78.28	69.78	64.56	61.21	58.86	56.93	55.39	54.09	54.19	24.09	61.48	63.57

Using the Incremental Few-shot Bodmas dataset as an example, several key observations can be made from the experimental results:

(1) Our method outperforms both the Finetune and CIL methods, which retain a small number of historical samples. Notably, it achieves better results without relying on historical samples.
(2) The proposed method also surpasses recent FSCIL methods, including GP-Tree [44], CSFCIL [56], CEC [48], and BFS-NID [57] by achieving the highest accuracy and the lowest performance drop across all sessions. Compared with the state-of-the-art method FACT [42], our approach demonstrates superior performance and achieves the lowest drop rate in most sessions.
(3) Specifically, our method, MalFSCIL, improved the final average accuracy over CEC [48] by 2.58% on Incremental Few-shot Bodmas dataset. Moreover, it outperformed the existing state-of-the-art method FACT [42] with a 1.85% improvement in final average accuracy.
(4) In the final session accuracy, our method surpassed the second-best method, CEC [48], by 2.68% and outperformed FACT [42] by 1.71%. These results highlight the effectiveness of our approach in learning more generalizable feature embeddings, rendering it a robust solution to the FSCIL problem.
(5) To ensure fairness among the comparison methods, we maintained the same backbone network as other methods and did not use the self-supervised pre-trained backbone network of the BFS-IDS method. We observed that the MalFSCL method outperformed BFS-IDS on both datasets from Table II and III. During the incremental learning phase, the performance of the BFS-IDS method was relatively weak due to severe data imbalance in each session of the Nsfocus dataset.
(6) We compared different methods on the micro-average ROC curve and AUC values for the last session of the Incremental Few-shot Bodmas dataset. As shown in Fig. 9, our method achieved the highest AUC value compared to other methods.

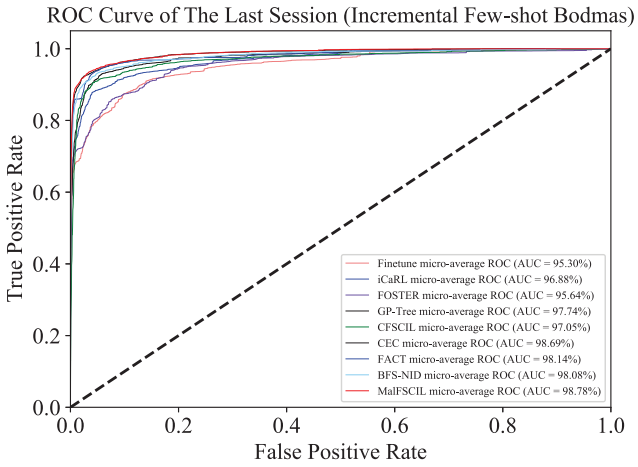

Fig. 9. Micro-average ROC curve and AUC values for the last session of the Incremental Few-shot Bodmas dataset.

C. Ablation Studies and Analysis

In this section, we present the results of extensive ablation studies conducted on both datasets to evaluate the impact of various components of the proposed method. Table IV and Table V summarizes the overall ablation results for both datasets.

Effect of reconstructing the learned samples. As shown in Table IV, the MalFSCIL-VAE model achieved a final average accuracy improvement of 2.12% and a final session accuracy improvement of 1.89% when using variational autoencoding to enhance the feature representation capabilities. Table V also shows that MalFSCIL-VAE achieved a final average accuracy improvement of 0.45% and a final session accuracy improvement of 0.62%. This demonstrates a clear advantage in the malware few-shot class incremental learning scenario. The method also performed well across all sessions, exhibiting the lowest performance drop rate. Thus, enhancing the model's ability to reconstruct learned samples improves feature embedding and generalization, leading to better overall performance.

Effects of the additive angular margin loss function. As shown in Table IV, the MalFSCIL-(VAE+ArcFace) model achieved a final average accuracy improvement of 2.58% and a

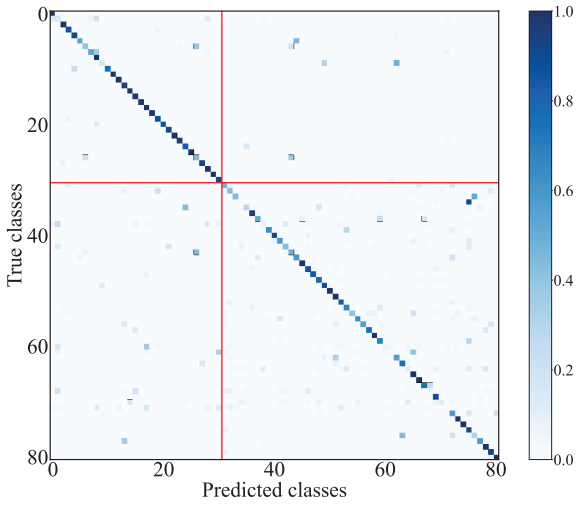

Fig. 10. Confusion matrix on Incremental Few-shot Bodmas dataset. We use bold red lines to separate regions of the base session classes and incremental classes of new sessions.

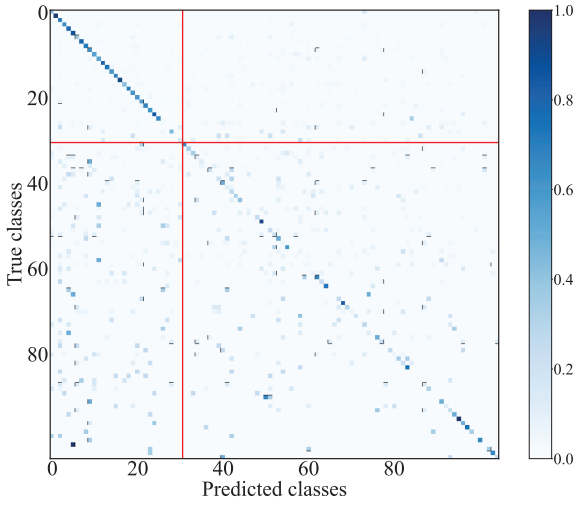

Fig. 11. Confusion matrix on NSFOCUS Real-World Malware dataset. We use bold red lines to separate regions of the base session classes and incremental classes of new sessions.

final session accurad y improvement of 2.48%. When incorporating the additive angular margin loss function, Table V also shows that the MalFSCIL-(VAE+ArcFace) model improved the final average accuracy by 0.76% and the final session accuracy by 0.86%. The method displayed good performance and low performance drop rates across most sessions. These results demonstrate that by applying the angular spacing penalty, the model compresses the angle range within each class while increasing the separation between classes. This ensures better intra-class compactness and inter-class separability, which is crucial for effectively training the GAT model. Consequently, when new classes are incrementally added, the unified classifier demonstrates improved classification performance for both old and new classes.

D. False Positives Analysis

In network security applications, a high number of false positives can lead to missed malware detections, resulting in stolen assets or data. Moreover, excessive false positives can increase the cost of manpower and time required for analysis. To address this, we present normalized confusion matrices to evaluate false positives in the final incremental learning tasks. As evident from Fig. 10 and Fig. 11, we report the confusion matrices for the Incremental Few-shot Bodmas Malware and NSFOCUS Real-World Malware datasets, with bold red lines separating the base session classes from the new classes of subsequent sessions. The x-axis of the confusion matrix represents the ground truth, while the y-axis represents the model's predictions. The diagonal elements of the confusion matrix indicate the model's classification accuracy.

Malware classification is crucial for identifying and differentiating the behavior, functions, and threat levels of various malware types, thereby enabling appropriate security measures and response strategies. Furthermore, malware classification assists in quickly identifying, tracking, and analyzing the changes and evolution of malware families or organizations, as well as assessing the risks and threats to network security.

Fig. 10 and Fig. 11 illustrate that most attack classes are distinguishable, but some exhibit significant confusion. For instance, in Fig. 10, attack samples from "backdoor.qukart" are misclassified as "backdoor.berbew" and "trojan.qukart," with false positive rates of 21% and 36%, respectively. Additionally, "dropper.gepys" samples are misclassified as "trojan.gepys" with a false positive rate of 36%, while "trojan.occamy" is misclassified as "trojan.smokeloader" with a 36% false positive rate.

We observe that the attack classes most prone to confusion tend to share the same type or family, making them harder to distinguish. For example, "backdoor.qukart" and "backdoor.berbew" are both of the "backdoor" type but belong to different families, while "backdoor.qukart" and "trojan.qukart" are part of the same "qukart" family but represent different types. Malware of the same type often shares similar function calls, leading to some degree of similarity. Likewise, malware within the same family tends to exhibit similar coding styles. When samples belong to the same family but differ in type, a mix of similar and dissimilar features can interfere with accurate classification.

As shown in Fig. 11, attack samples from the "POISON CARP" organization are misclassified as "Greenbug" with a false positive rate of 67%. Similarly, "Bahamut" organization samples are misclassified as "Viceroy Tiger" with a 100% false positive rate. It is notable that both the Bahamut and Viceroy Tiger organizations focus on data theft as a primary attack objective, which may contribute to misclassification due to overlapping characteristics. The high-resolution figures have been moved to the end of this paper to provide a clearer presentation of the results.

VI. Discussion

As the first attempt at few-shot class-incremental malware detection in a deep open-world setting, our approach demonstrates several advantages as well as limitations. In this section, we discuss these strengths, acknowledge the limitations, and suggest potential directions for future research.

TABLE VI

RESULT ANALYSIS OF FEW-SHOT SAMPLE INCREMENTAL LEARNING ON INCREMENTAL FEW-SHOT BODMAS. ◇ INDICATES FEW-SHOT SAMPLE INCREMENTAL LEARNING WITH DRIFT IN KNOWN CLASS SAMPLES

Method	Accuracy in each session (%)											PD	Average Acc	AUT
	0	1	2	3	4	5	6	7	8	9	10			
MalFSCIL	90.58	87.85	87.65	84.27	83.53	82.63	81.51	79.49	78.90	78.26	78.05	12.53	82.97	82.32
MalFSCIL◇	90.58	87.97	87.65	84.27	83.53	82.63	81.51	79.49	78.90	78.26	78.05	12.93	82.99	82.32

TABLE VII

COMPARISON OF DIFFERENT METHODS. ALL EXPERIMENTS RUN ON NVIDIA TESLA V100. FLOPS ARE TESTED WITH ONE SAMPLE AND THE INFERENCE SPEED (SAMPLES/SECOND) IS THE AVERAGE OF 100 RUNS WITH A BATCH SIZE OF 100 AFTER HARDWARE WARM-UP

Model	FLOPs	Params	Inference Speed	
			Incremental Few-shot Bodmas	NSFOCUS Real-World Malware
Finetune	1065.01M	11.17M	2179	2846
iRaRL	1065.01M	11.17M	2862	2092
FOSTER	2130.02M	22.34M	1707	1478
GP-Tree	297.83M	11.46M	601	376
CFSCIL	3523.35M	12.75M	1958	4189
CEC	297.55M	11.18M	1816	2522
FACT	297.55M	11.18M	3321	4259
BFS-NID	149.12M	11.44M	978	1813
MalFSCIL	557.88M	11.17M	2560	2761

A. Scalability

Thus far, our evaluation has primarily focused on incremental learning from previously unseen malware classes (few-shot class-incremental learning), without addressing the automatic discovery of new malware families from detected unknown class samples. In our current setup, training samples for new families in each incremental session are manually labeled by security analysts. However, the automatic discovery of new malware families is an important and valuable research direction. Automating this process would not only transform MalFSCIL into a fully automated deep open-world malware recognition system but also substantially reduce the cost and effort associated with manual annotation. In future work, we plan to integrate unsupervised learning methods, such as contrastive clustering, to automatically identify new malware families from samples of unknown classes. This enhancement would significantly improve the system's scalability and level of automation, making it a more robust solution for real-world malware detection scenarios.

B. Computational Complexity

We compared MalFSCIL with baseline algorithms in terms of parameter count and floating-point operations per second (FLOPs), as shown in Table VII. Despite the improvements in detection performance, MalFSCIL does not significantly increase computational complexity compared to some of the other methods. This efficiency is primarily due to our decoupled training framework: after completing the training in the base class session, the parameters of the feature extraction module are frozen, and only the class-prototype classifier is trained. This strategy effectively reduces the computational load during gradient updates, thereby enhancing training efficiency. Although the computational time during the testing phase increases slightly, the overall impact remains minimal. Future research will focus on exploring model compression and acceleration techniques, such as knowledge distillation and model pruning, to further optimize computational complexity. These efforts aim to improve MalFSCIL's operational efficiency and make it more suitable for deployment in real-world applications.

C. Adaptability

MalFSCIL is designed to capture the dynamic evolution of malware through few-shot incremental learning. However, when faced with highly diverse malware that employs advanced obfuscation and evasion strategies, the model may encounter concept drift issues, leading to reduced generalization and unstable recognition performance. To evaluate the model's adaptability to drift samples, we conducted specific experiments. After performing incremental learning on a few categories, we introduced a small number of drift samples from other malware categories to continuously update the model's understanding of malware attacks. The experimental results (see Table VI) demonstrate that adding only five drift samples per family for the "trojan.vflooder", "trojan.zbot", "trojan.lunam", "backdoor.bladabindi", and "trojan.gandcrab" significantly improves classification performance, indicating that MalFSCIL effectively handles concept drift.

Notably, as a few-shot class-incremental learning algorithm, MalFSCIL leverages core mechanisms such as data reshaping using Variational Autoencoders (VAEs) and dynamic boundary partitioning methods. These features enable the model to incrementally update with a small number of new class samples while mitigating the risk of forgetting previously learned information. The framework is highly generalizable and, when applied to anomaly detection tasks involving network traffic, log data, or user behavior, can still effectively detect new network attacks or anomalous behaviors by adjusting its architecture and parameters. This adaptability allows MalFSCIL to excel not only in malware detection but also in broader cybersecurity applications.

VII. CONCLUSION

Deep learning-based models typically operate under a "closed-world" assumption, expecting the testing data distribution to closely mirror that of the training data. However, in the evolving landscape of cybersecurity, the continuous emergence of new attack methods and types—driven by the attack-defense confrontation—causes data distributions to drift over time. This concept drift can result in critical errors for

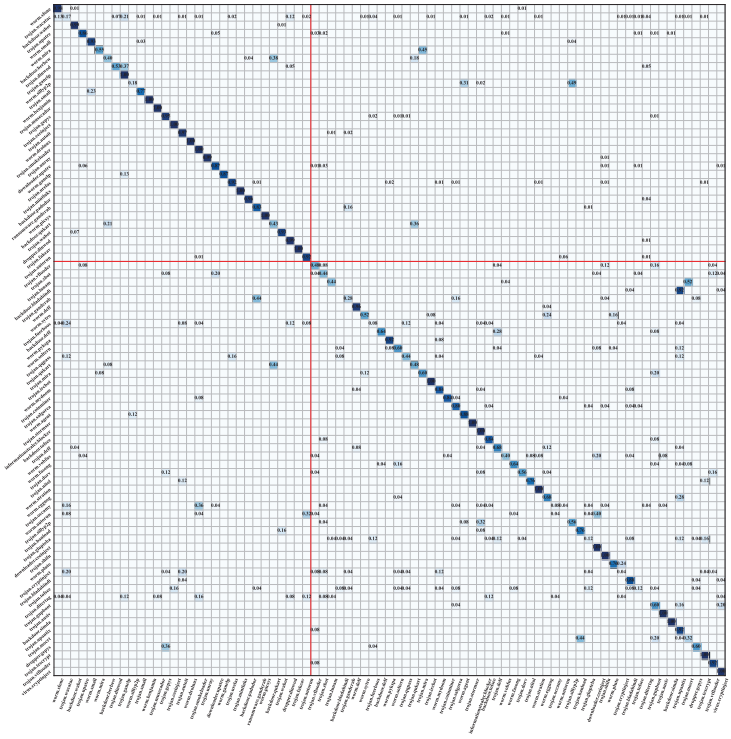

Fig. 12. Normalized confusion matrices on Incremental Few-shot Bodmas Malware dataset. We use bold red lines to separate regions of the base session classes and incremental classes of new sessions.

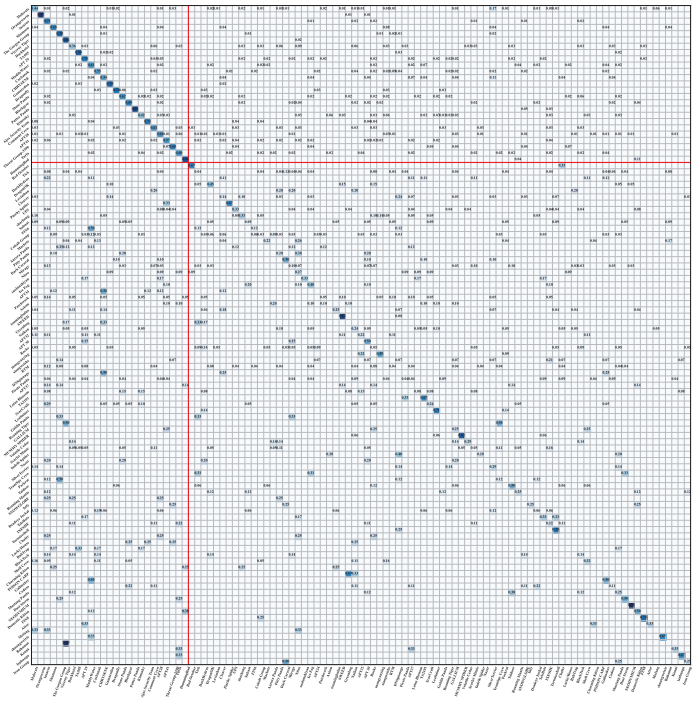

Fig. 13. Normalized confusion matrices on Incremental Few-shot APT Malware dataset. We use bold red lines to separate regions of the base session classes and incremental classes of new sessions.

detection models. To detect malware more effectively and in a timely manner, a practical malware detection model must be capable of incrementally learning new attack knowledge, even when only a small number of samples are available, while retaining previously learned knowledge. In this work, we proposed a few-shot class-incremental learning framework for malware detection based on a decoupling strategy that separates "old knowledge reshaping" from "new knowledge increment." Feature enhancement and data augmentation techniques were applied during both the base class training stage and the new class adaptation stage to address the issue of catastrophic forgetting. This enables the malware detection

model to continuously learn and update in few-shot scenarios. Experimental results demonstrated that our approach outperformed state-of-the-art methods across all benchmark datasets, achieving substantial improvements in detection performance.

APPENDIX

See Figs. 12 and 13.

REFERENCES

[1] X. Ling et al., "Adversarial attacks against windows PE malware detection: A survey of the state-of-the-art," *Comput. Secur.*, vol. 128, May 2023, Art. no. 103134.

[2] N. J. Palatty. (2022). *Statistics 2024: Trends & Cost*. [Online]. Available: https://www.getastra.com/blog/security-audit/ransomware-attack-statistics/

[3] S. Cesare, Y. Xiang, and W. Zhou, "Control flow-based malware variant detection," *IEEE Trans. Dependable Secur. Comput.*, vol. 11, no. 4, pp. 307–317, Jul. 2014.

[4] J. Yu, Y. He, Q. Yan, and X. Kang, "SpecView: Malware spectrum visualization framework with singular spectrum transformation," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 5093–5107, 2021.

[5] S. E. Coull and C. Gardner, "Activation analysis of a byte-based deep neural network for malware classification," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2019, pp. 21–27.

[6] D. Gibert, C. Mateu, and J. Planes, "HYDRA: A multimodal deep learning framework for malware classification," *Comput. Secur.*, vol. 95, Aug. 2020, Art. no. 101873.

[7] W. Fang et al., "Comprehensive Android malware detection based on federated learning architecture," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 3977–3990, 2023.

[8] R. Liu and C. Nicholas, "IMCDCF: An incremental malware detection approach using hidden Markov models," 2023, *arXiv:2304.07989*.

[9] Q. Qiang et al., "An incremental malware classification approach based on few-shot learning," in *Proc. IEEE Int. Conf. Commun.*, May 2022, pp. 2682–2687.

[10] I. Santos, F. Brezo, X. Ugarte-Pedrero, and P. G. Bringas, "Opcode sequences as representation of executables for data-mining-based unknown malware detection," *Inf. Sci.*, vol. 231, pp. 64–82, May 2013.

[11] T. Lee, B. Choi, Y. Shin, and J. Kwak, "Automatic malware mutant detection and group classification based on the n-gram and clustering coefficient," *J. Supercomput.*, vol. 74, no. 8, pp. 3489–3503, Aug. 2018.

[12] H. Kim, J. Kim, Y. Kim, I. Kim, K. J. Kim, and H. Kim, "Improvement of malware detection and classification using API call sequence alignment and visualization," *Cluster Comput.*, vol. 22, no. S1, pp. 921–929, Jan. 2019.

[13] J. Yu-Chin Cheng, T.-S. Tsai, and C.-S. Yang, "An information retrieval approach for malware classification based on windows API calls," in *Proc. Int. Conf. Mach. Learn. Cybern.*, vol. 4, Jul. 2013, pp. 1678–1683.

[14] S. Sebastián and J. Caballero, "AVclass2: Massive malware tag extraction from AV labels," in *Proc. Annu. Comput. Security Appl. Conf.*, 2020, pp. 42–53.

[15] B. Alsulami, A. Srinivasan, H. Dong, and S. Mancoridis, "Lightweight behavioral malware detection for windows platforms," in *Proc. 12th Int. Conf. Malicious Unwanted Softw. (MALWARE)*, Oct. 2017, pp. 75–81.

[16] X. Zhang et al., "Slowing down the aging of learning-based malware detectors with API knowledge," *IEEE Trans. Dependable Secur. Comput.*, vol. 20, no. 2, pp. 902–916, Mar. 2023.

[17] H. Peng et al., "MalGNE: Enhancing the performance and efficiency of CFG-based malware detector by graph node embedding in low dimension space," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4881–4896, 2024.

[18] J. Ning et al., "Malware traffic classification using domain adaptation and ladder network for secure industrial Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 17058–17069, Sep. 2022.

[19] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3431–3446.

[20] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, "New directions in automated traffic analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3366–3383.

[21] Y. He, X. Kang, Q. Yan, and E. Li, "ResNeXt+: Attention mechanisms based on ResNeXt for malware detection and classification," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1142–1155, 2024.

[22] B. Yuan, J. Wang, D. Liu, W. Guo, P. Wu, and X. Bao, "Byte-level malware classification based on Markov images and deep learning," *Comput. Secur.*, vol. 92, May 2020, Art. no. 101740.

[23] T. Manhar Mohammed, L. Nataraj, S. Chikkagoudar, S. Chandrasekaran, and B. S. Manjunath, "Malware detection using frequency domain-based image visualization and deep learning," 2021, *arXiv:2101.10578*.

[24] O. Barut, Y. Luo, P. Li, and T. Zhang, "R1DIT: Privacy-preserving malware traffic classification with attention-based neural networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 2071–2085, Jun. 2023.

[25] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. 34th Int. Conf. Mach. Learn.*, vol. 70, Aug. 2017, pp. 1126–1135.

[26] C. Liu, B. Li, J. Zhao, W. Feng, X. Liu, and C. Li, "A2-CLM: Few-shot malware detection based on adversarial heterogeneous graph augmentation," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 2023–2038, 2024.

[27] B. Liu et al., "Negative margin matters: Understanding margin in few-shot classification," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 438–455.

[28] Z. Tang, P. Wang, and J. Wang, "ConvProtoNet: Deep prototype induction towards better class representation for few-shot malware classification," *Appl. Sci.*, vol. 10, no. 8, p. 2847, Apr. 2020.

[29] P. Wang, Z. Tang, and J. Wang, "A novel few-shot malware classification approach for unknown family recognition with multi-prototype modeling," *Comput. Secur.*, vol. 106, Jul. 2021, Art. no. 102273.

[30] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.

[31] L. Yu et al., "Semantic drift compensation for class-incremental learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recog. (CVPR)*, May 2020, pp. 13–19.

[32] J. Li, D. Xue, W. Wu, and J. Wang, "Incremental learning for malware classification in small datasets," *Secur. Commun. Netw.*, vol. 2020, pp. 1–12, Feb. 2020.

[33] T. Lu and J. Wang, "DOMR: Toward deep open-world malware recognition," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1455–1468, 2024.

[34] F. M. Castro, M. J. Marín-Jiménez, N. Guil, C. Schmid, and K. Alahari, "End-to-end incremental learning," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 233–248.

[35] X. Hu, K. Tang, C. Miao, X.-S. Hua, and H. Zhang, "Distilling causal effect of data in class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 3956–3965.

[36] X. Tao, X. Hong, X. Chang, S. Dong, X. Wei, and Y. Gong, "Few-shot class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 12183–12192.

[37] M. Ren, R. Liao, E. Fetaya, and R. S. Zemel, "Incremental few-shot learning with attention attractor networks," *Adv. Neural Inform. Process. Syst. (NeurIPS)*, vol. 32, pp. 5275–5285, Oct. 2019.

[38] B. Fritzke, "A growing neural gas network learns topologies," *Adv. Neural Inform. Process. Syst. (NeurIPS)*, vol. 7, pp. 625–632, Jan. 1994.

[39] P. Mazumder, P. Singh, and P. Rai, "Few-shot lifelong learning," *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 3, pp. 2337–2345, May 2021.

[40] K. Zhu, Y. Cao, W. Zhai, J. Cheng, and Z.-J. Zha, "Self-promoted prototype refinement for few-shot class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 6801–6810.

[41] A. Cheraghian, S. Rahman, P. Fang, S. K. Roy, L. Petersson, and M. Harandi, "Semantic-aware knowledge distillation for few-shot class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 2534–2543.

[42] D.-W. Zhou, F.-Y. Wang, H.-J. Ye, L. Ma, S. Pu, and D.-C. Zhan, "Forward compatible few-shot class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 9046–9056.

[43] S. Dong, X. Hong, X. Tao, X. Chang, X. Wei, and Y. Gong, "Few-shot class-incremental learning via relation knowledge distillation," in *Proc. AAAI*, 2021, pp. 1255–1263.

[44] I. Achituve, A. Navon, Y. Yemini, G. Chechik, and E. Fetaya, "GP-Tree: A Gaussian process classifier for few-shot incremental learning," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 54–65.

[45] D.-W. Li and H. Huang, "Few-shot class-incremental learning via compact and separable features for fine-grained vehicle recognition," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 21418–21429, Nov. 2022.

[46] D. P Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, *arXiv:1312.6114*.

[47] Y. Nagano, R. Karakida, and M. Okada, "Collective dynamics of repeated inference in variational autoencoder rapidly find cluster structure," *Sci. Rep.*, vol. 10, no. 1, p. 16001, Sep. 2020.
[48] C. Zhang, N. Song, G. Lin, Y. Zheng, P. Pan, and Y. Xu, "Few-shot incremental learning with continually evolved classifiers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 12455–12464.
[49] P. Veličkovi'c, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
[50] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4690–4699.
[51] L. Yang, A. Ciptadi, I. Laziuk, A. Ahmadzadeh, and G. Wang, "BODMAS: An open dataset for learning based temporal analysis of PE malware," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2021, pp. 78–84.
[52] (2022). *Malware Dataset*. [Online]. Available: https://nsfocusglobal.com/
[53] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "s$*TESSERACT*$: Eliminating experimental bias in malware classification across space and time," in *Proc. USENIX Secur. Symp.*, 2019, pp. 729–746.
[54] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "ICaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2001–2010.
[55] F. Y. Wang, D. W. Zhou, H. J. Ye, and D.-C. Zha, "FOSTER: Feature boosting and compression for class-incremental learning," in *Proc. 17th Eur. Conf. Comput. Vision (ECCV)*, vol. 13685, 2022, pp. 398–414.
[56] M. Hersche, G. Karunaratne, G. Cherubini, L. Benini, A. Sebastian, and A. Rahimi, "Constrained few-shot class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 9057–9067.
[57] L. Du, Z. Gu, Y. Wang, L. Wang, and Y. Jia, "A few-shot class-incremental learning method for network intrusion detection," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 2, pp. 2389–2401, Apr. 2024.
[58] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

Yuhan Chai received the Ph.D. degree from the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China, in 2023. She is currently a Post-Doctoral Researcher with the Cyberspace Institute of Advanced Technology, Guangzhou University. Her research interests include cybersecurity, malware detection, and software supply chain security.

Ximing Chen is currently pursuing the Ph.D. degree with the Cyberspace Institute of Advanced Technology, Guangzhou University, China. His current research interests include network security risk assessment and malicious traffic detection.

Jing Qiu (Senior Member, IEEE) received the Ph.D. degree in computer applications technology from Beijing Institute of Technology in 2010. She was a Visiting Scholar with the University of Southern California, LA, USA. She is currently a Professor and the Ph.D. Supervisor of the Cyberspace Institute of Advanced Technology, Guangzhou University. She is also a part-time Researcher with the Peng Cheng Laboratory, Shenzhen, China. She is a Distinguished Member of China Computer Federation.

Lei Du received the master's degree from Hebei University of Science and Technology in 2021. He is currently with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. He is also with the Peng Cheng Laboratory, Shenzhen. His research interests include cyberspace security, machine intelligence, and open-world learning.

Yanjun Xiao is currently working with PINGXING Lab (Nsfocus Technology Group Company). His current research interests include situation awareness, knowledge graph, APT tracking, artificial intelligence decision-making and command, and cyber range.

Qiying Feng received the B.S. degree in electronic information engineering from South China Agricultural University, Guangzhou, China, in 2014, and the Ph.D. degree in computer and information science from the University of Macau, Macau, China, in 2020. From 2020 to 2024, she worked as a Post-Doctoral Fellow with the South China University of Technology, Guangzhou. She is currently an Associate Professor with Guangzhou University. Her research interests include soft computing and machine learning.

Shouling Ji (Member, IEEE) received the Ph.D. degree in electrical and computer engineering from Georgia Institute of Technology and the Ph.D. degree in computer science from Georgia State University. He is currently a ZJU 100-Young Professor with the College of Computer Science and Technology, Zhejiang University, and a Research Faculty Member with the School of Electrical and Computer Engineering, Georgia Institute of Technology. His current research interests include AI security, data-driven security, privacy, and data analytics. He is a member of ACM and was the Membership Chair of the IEEE Student Branch with Georgia State University from 2012 to 2013.

Zhihong Tian (Senior Member, IEEE) is currently a Professor with the Cyberspace Institute of Advanced Technology and the Vice President of Guangzhou University, Guangzhou, Guangdong, China, and Guangdong Province Universities, and a Distinguished Professor with the College Pearl River Scholar. He is also a part-time Professor with Carlton University, Ottawa, ON, Canada. Previously, he served in different academic and administrative positions with Harbin Institute of Technology, Harbin, China. He has authored over 200 journal and conference papers in these areas. His research interests include computer networks and cyberspace security. His research has been supported in part by the National Natural Science Foundation of China, the National Key Research and Development Plan of China, the National High-Tech Research and Development Program of China (863 Program), and the National Basic Research Program of China (973 Program).